

mongodb-hands-on

MongoDB - Hands-On Concepts

MongoDB is a NoSQL database that stores data as documents (JSON-like objects) instead of rows and tables. Understanding how to read, write, and query data correctly is the foundation for working with it confidently.

1. Core Concepts to Understand First

Term	What it means
Database	A container that holds collections (similar to a SQL database)
Collection	A group of documents (similar to a SQL table)
Document	A single record stored as a JSON-like object
Field	A key-value pair inside a document
<code>_id</code>	A unique identifier automatically added to every document

2. Connecting via Shell

```
mongosh
```

Switch to a database (creates it automatically if it does not exist):

```
use handsOn
```

3. CRUD Operations

These are the four operations you will use constantly.

Create

```
// Insert one document
db.users.insertOne({ name: "Alice", age: 25, role: "intern" })

// Insert multiple documents
db.users.insertMany([
  { name: "Bob", age: 22, role: "intern" },
  { name: "Carol", age: 28, role: "developer" }
])
```

Read

```
// Get all documents in a collection
db.users.find()

// Filter by a field
db.users.find({ role: "intern" })

// Find one document
db.users.findOne({ name: "Alice" })
```

```
// Only return specific fields (1 = include, 0 = exclude)
db.users.find({ role: "intern" }, { name: 1, age: 1, _id: 0 })
```

Update

```
// Update one document
db.users.updateOne(
  { name: "Alice" },
  { $set: { age: 26 } }
)

// Update all matching documents
db.users.updateMany(
  { role: "intern" },
  { $set: { status: "active" } }
)
```

Delete

```
// Delete one document
db.users.deleteOne({ name: "Bob" })

// Delete all matching documents
db.users.deleteMany({ role: "developer" })
```

Note: Insert few documents before proceeding as we performed deletion

4. Querying - Filters and Operators

```
// Greater than / Less than
db.users.find({ age: { $gt: 20 } }) // age > 20
db.users.find({ age: { $lt: 30 } }) // age < 30
db.users.find({ age: { $gte: 22, $lte: 28 } }) // between 22 and 28

// Match multiple conditions (AND)
db.users.find({ role: "intern", age: { $gte: 22 } })

// Match either condition (OR)
db.users.find({ $or: [{ role: "intern" }, { role: "developer" }] })

// Field exists check
db.users.find({ status: { $exists: true } })
```

5. Sorting, Limiting, and Skipping

```
// Sort ascending (1) or descending (-1)
db.users.find().sort({ age: 1 })
db.users.find().sort({ age: -1 })

// Limit results
db.users.find().limit(5)
```

```
// Skip results (useful for pagination)
db.users.find().skip(1).limit(1)
```

6. Indexes

Indexes make queries faster. Without them, MongoDB scans every document in a collection.

```
// Create an index on a single field
db.users.createIndex({ name: 1 })

// Dropping created index
db.users.dropIndex({ name: 1 })

// Create a unique index (no duplicate values allowed)
db.users.createIndex({ name: 1 }, { unique: true })

// View all indexes on a collection
db.users.getIndexes()
```

Always index fields you query or filter by frequently.

7. Aggregation Pipeline

Aggregation is used when you need to process and transform data - grouping, counting, calculating averages, etc.

```
// Count documents grouped by role
db.users.aggregate([
  { $group: { _id: "$role", count: { $sum: 1 } } }
])

// Filter, then group
db.users.aggregate([
  { $match: { status: "active" } },
  { $group: { _id: "$role", averageAge: { $avg: "$age" } } }
])

// Sort after grouping
db.users.aggregate([
  { $group: { _id: "$role", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
```

Each step in the array is a stage - data flows from one stage to the next.

8. Useful Utility Commands

```
// List all databases
show dbs

// List all collections in the current database
show collections

// Count documents in a collection
```

```
db.users.countDocuments()
db.users.countDocuments({ role: "intern" })

// Drop a collection (permanent - use carefully)
db.users.drop()

// Drop the entire database (permanent - use carefully)
db.dropDatabase()
```

9. Load Data from a JSON File

When you have a larger dataset as a `.json` file, use `mongoimport` instead of inserting manually.

The JSON file must be an array of objects:

```
[
  { "name": "Alice", "age": 29, "role": "Engineer", "department": "Engineering", "salary": 95000, "company_id": "c1", "status": "active" },
  { "name": "Bob", "age": 35, "role": "Manager", "department": "Engineering", "salary": 120000, "company_id": "c1", "status": "active" }
]
```

Run this from your terminal (not inside mongosh):

```
mongoimport --db company_db --collection employees --file employees.json --jsonArray
```

10. Things Worth Knowing

- MongoDB does not enforce a schema by default. Two documents in the same collection can have completely different fields. This is flexible but requires discipline.
- Always use `$set` inside `updateOne` / `updateMany`. Without it, the entire document gets replaced.
- Avoid storing large blobs (images, files) directly in documents. Keep documents small.